Studio Nx

백엔드 276 — 클립 · 연결 스키마

2026-10-19

백엔드 276 — 클립 · 연결 스키마 (schemas/clip.py · schemas/connection.py)

항목	내용
파일	`studionx/backend/app/schemas/clip.py`, `studionx/backend/app/schemas/connection.py`
작업	S1-10a 클립·연결·패턴 API: 클립과 연결의 요청과 응답 스키마 (PRD 02 FR-2196, FR-2197)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 275 `TrackConnection` 모델, 다음 S1-10a 의 서비스와 라우터 문서

1. 하는 일

클립 API 와 연결 API 가 주고받는 요청과 응답의 모양을 정합니다. 요청 칸의 범위를 검사해 잘못된 값이 DB 에 닿기 전에 422 로 막고, 응답은 DB 행을 그대로 되돌려 줍니다.

```
POST   /tracks/{track_id}/clips       ClipCreateRequest         → ClipResponse (201)
PATCH  /clips/{clip_id}               ClipUpdateRequest         → ClipResponse
PUT    /tracks/{track_id}/connection  TrackConnectionSetRequest → TrackConnectionResponse
GET    /projects/{id}/full            한 번의 조회              → clips 목록, 트랙별 connections
```

주요 검사는 다음과 같습니다.

칸	검사	뜻
`name`	앞뒤 공백을 지우고 1자 이상 100자 이하	DB `VARCHAR(100)` 과 같은 길이입니다
`start_tick`	0 이상	타임라인 시작 위치라 음수가 될 수 없습니다
`length_tick`	1 이상	길이가 0인 클립은 없으므로 0 은 422 로 거절됩니다
`offset_tick`, `fade_in_tick`, `fade_out_tick`	0 이상	트림 위치와 페이드 길이는 음수가 아닙니다
`gain_db`, `volume_db`	-60 이상 24 이하	허용하는 dB 범위입니다
`pan`	-1 이상 1 이하	왼쪽 끝부터 오른쪽 끝까지입니다
`fade_in_curve`, `fade_out_curve`	-1 이상 1 이하	DB 의 `chk_clips_fade_curve` 와 같습니다
`pitch_semitones`	-48 이상 48 이하	DB 의 `chk_clips_pitch` 와 같습니다
`pattern_id`, `sample_id`, `to_track_id`, `to_track_plugin_id`	1 이상	번호 칸이라 0 또는 음수를 받지 않습니다
`to_port`	1 이상 16 이하	받는 트랙의 입력 번호 범위입니다
`clip_type`, `target`, `warp_mode`	`Literal` 의 값만	DB ENUM 과 같은 글자만 통과합니다

2. 코드

요청 (`clip.py`)

```
class ClipCreateRequest(BaseModel):
    """POST /tracks/{id}/clips — 타임라인에 클립을 놓는다.

    MIDI 클립은 pattern_id 를, AUDIO 클립은 sample_id 를 반드시 보낸다 (반대쪽은 비운다).
    """

    clip_type: ClipType
    pattern_id: int | None = Field(default=None, ge=1)
    sample_id: int | None = Field(default=None, ge=1)
    name: ClipName | None = None
    start_tick: int = Field(ge=0)
    length_tick: int = Field(ge=1)
    offset_tick: int = Field(default=0, ge=0)
    gain_db: Decimal = Field(default=Decimal("0.00"), ge=-60, le=24)
    is_looped: bool = False
    fade_in_tick: int = Field(default=0, ge=0)
    fade_out_tick: int = Field(default=0, ge=0)
    fade_in_curve: Decimal = Field(default=Decimal("0.00"), ge=-1, le=1)
    fade_out_curve: Decimal = Field(default=Decimal("0.00"), ge=-1, le=1)
    warp_mode: WarpMode = "BEATS"
    warp_markers: dict[str, Any] | None = None
    pitch_semitones: Decimal = Field(default=Decimal("0.00"), ge=-48, le=48)
    is_reversed: bool = False

    @model_validator(mode="after")
    def _kind_matches_ids(self) -> Self:
        """MIDI 는 pattern_id 만, AUDIO 는 sample_id 만 — DB CHECK 와 같은 규칙."""
        if self.clip_type == "MIDI":
            if self.pattern_id is None or self.sample_id is not None:
                raise ValueError("MIDI clip needs pattern_id only")
        elif self.sample_id is None or self.pattern_id is not None:
            raise ValueError("AUDIO clip needs sample_id only")
        return self
```

요청 (`connection.py`)

```
class TrackConnectionSetRequest(BaseModel):
    """PUT /tracks/{id}/connection — 이 트랙에서 나가는 연결을 통째로 맞춘다.

    to_track_id 는 받는 트랙이다. target 이 PLUGIN 이면 to_track_plugin_id 로 그 트랙의
    이펙트 한 칸을 고른다(없으면 트랙 입력). INSTRUMENTS 면 악기 입력이라 이펙트는 없다.
    """

    to_track_id: int = Field(ge=1)
    target: ConnectionTarget = "PLUGIN"
    to_track_plugin_id: int | None = Field(default=None, ge=1)
    to_port: int = Field(default=1, ge=1, le=16)
    also_play: bool | None = None
    volume_db: Decimal = Field(default=Decimal("0.00"), ge=-60, le=24)
    pan: Decimal = Field(default=Decimal("0.00"), ge=-1, le=1)
    is_solo: bool = False
```

나머지 세 클래스는 칸만 봐도 알 수 있습니다. `ClipUpdateRequest` 는 모든 칸이 `| None` 이라서 `clip_service` 가 `exclude_unset=True` 로 보낸 칸만 고치고, 종류와 `pattern_id`, `sample_id` 는 아예 넣지 않았습니다. 다른 종류의 클립으로 바꾸는 일은 없기 때문입니다. `ClipResponse` 는 `ConfigDict(from_attributes=True)` 로 DB 행을 그대로 받아 `clip_id` 부터 `updated_at` 까지 20개 칸을 돌려주고, `created_at` 은 제외합니다. `TrackConnectionResponse` 는 같은 방식으로 `connection_id` 부터 `is_solo` 까지 10개 칸을 돌려줍니다.

문법 풀이

식	뜻
`ClipType = Literal["AUDIO", "MIDI"]`	`AUDIO` 와 `MIDI` 두 글자만 허용하고, DB ENUM 과 같은 값을 씁니다
`ClipName = Annotated[str, StringConstraints(strip_whitespace=True, min_length=1, max_length=100)]`	이름의 앞뒤 공백을 지우고 1자 이상 100자 이하로 잡습니다
`CLIP_NAME_MAX = 100`	`clips.name` 칸 길이(`VARCHAR(100)`)와 맞춘 상수입니다
`Field(ge=0)`, `Field(ge=1)`, `Field(ge=-60, le=24)`	값이 몇 이상, 몇 이하인지 적습니다. 범위를 벗어나면 422 가 납니다
`@model_validator(mode="after")`	칸을 다 채운 뒤 한 번 더 검사합니다. 실패하면 FastAPI 가 422 로 바꿉니다
`raise ValueError("MIDI clip needs pattern_id only")`	규칙에 어긋난 요청을 막는 메시지입니다
`int | None = Field(default=None, ge=1)`	안 보내면 없음이고, 보내면 1 이상인 번호입니다
`warp_markers: dict[str, Any] | None`	모양이 정해지지 않은 값을 그대로 주고받습니다
`ConfigDict(from_attributes=True)`	ORM 행을 응답으로 그대로 바꿀 수 있게 합니다

3. 왜 이렇게

선택	이유
종류별 규칙을 `model_validator` 로 검사	필드 하나로는 MIDI 면 `pattern_id` 필수 같은 짝을 표현할 수 없습니다. 클래스를 다 만든 뒤 한 번 보고, DB 의 `chk_clips_kind` 와 같은 규칙을 422 로 막습니다
수정 요청의 모든 칸을 `None` 기본값으로	보낸 칸만 바꾸는 PATCH 와 맞습니다
종류와 `pattern_id`, `sample_id` 를 수정 요청에서 제외	다른 패턴이나 샘플로 바꾸는 일은 없습니다. 바꾸려면 새 클립을 만듭니다
응답에 `created_at` 를 빼고 `updated_at` 만 담음	화면이 필요로 하는 것은 마지막 수정 시각뿐입니다. `tests/test_clip_routes.py` 의 `CLIP_KEYS` 20개 칸과 맞습니다
연결 저장을 POST 와 DELETE 가 아니라 PUT 으로	보내는 트랙마다 행이 하나뿐이라(DB `uq_connections_from`) 통째로 맞추는 편이 표 규칙과 맞습니다
`to_track_id` 와 `to_track_plugin_id` 를 나눔	`target` 이 INSTRUMENTS 면 이펙트 칸이 필요 없습니다. 서비스가 INSTRUMENTS 일 때 이펙트 칸을 비웁니다
자기 자신, 서로, 고리 검사는 스키마에서 제외	두 트랙 사이의 관계는 표 한 줄로 알 수 없습니다. `connection_service` 가 400 으로 막습니다

4. 검증

`tests/test_clip_routes.py` 에서 `test_create_clip_validates_kind` 가 MIDI 인데 `pattern_id` 가 없는 경우, `sample_id` 까지 갖춘 경우, `length_tick` 이 0인 경우를 모두 422 로 확인하고, 응답 칸은 `CLIP_KEYS` 20개로 검사합니다. `tests/test_connection_routes.py` 에서 `test_put_creates_and_get_returns` 와 `test_put_replaces_existing` 은 PUT 통째로 맞추기, `test_put_instruments_target_drops_plugin` 은 INSTRUMENTS 일 때 이펙트 칸이 비워지는 것, `test_put_rejects_self_connection`, `test_put_rejects_mutual_connection`, `test_put_rejects_loop` 은 잘못된 연결의 400 을 확인합니다. `pytest -q` 는 1083 passed, `ruff check .` 는 오류 없이 통과합니다.

5. 남은 일 (사용자)

확인할 것	내용
문서 검토	274 와 275 문서와 함께 S1-10a 작업 단위가 끝난 뒤 한 번에 검토합니다
`/docs` 에서 확인	`POST /tracks/{id}/clips` 에 `length_tick: 0` 을 넣으면 422 가 나는지 눌러 볼 수 있습니다